fit@hcmus

Object-Oriented Programming

Design Pattern



Tran Duy Hoang

Design patterns

- A pattern is a description of the problem and the essence of its solution
- It should be sufficiently abstract to be reused in different settings
- Pattern descriptions usually make use of object-oriented characteristics such as inheritance and polymorphism.

Pattern elements

- Name
  - A meaningful pattern identifier.
- Problem description
- Solution description
  - Not a concrete design but a template for a design solution that can be instantiated in different ways
- Consequences
  - The results and trade-offs of applying the pattern

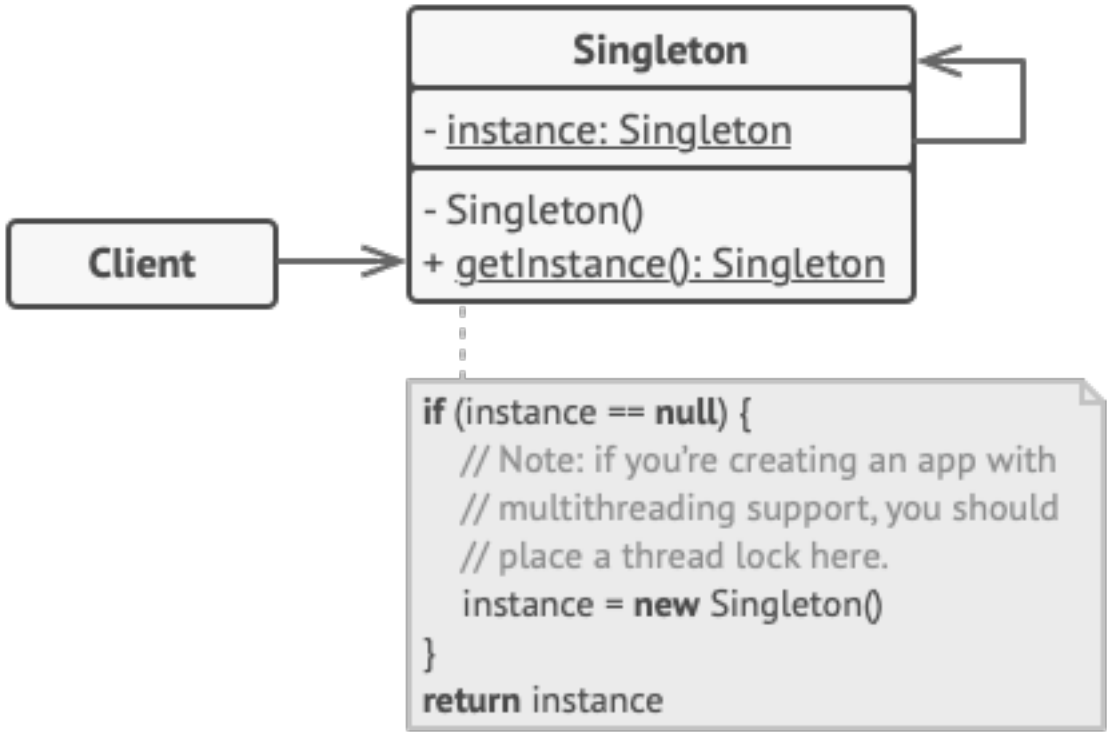
Purpose of Design patterns

- They are tried-and-true solutions
- They are simple to re-use
- They have a strong personalities
- They facilitate communication
- They eliminate the need for code refactoring
- They reduce the codebase's size

Singleton

- Intent
  - Ensure that a class has only one instance, while providing a global access point to this instance
- Problem
  1. Ensure that a class has just a single instance
  2. Provide a global access point to that instance
- Solution
  - Make the default constructor private
  - Create a static creation method that acts as a constructor

Singleton




```
public class Config {

    private static Config instance;

    private Config() {
        // Some initialization code
    }

    public static Config getInstance() {
        if (instance == null) {
            instance = new Config();
        }
        return instance;
    }
}
```

Singleton

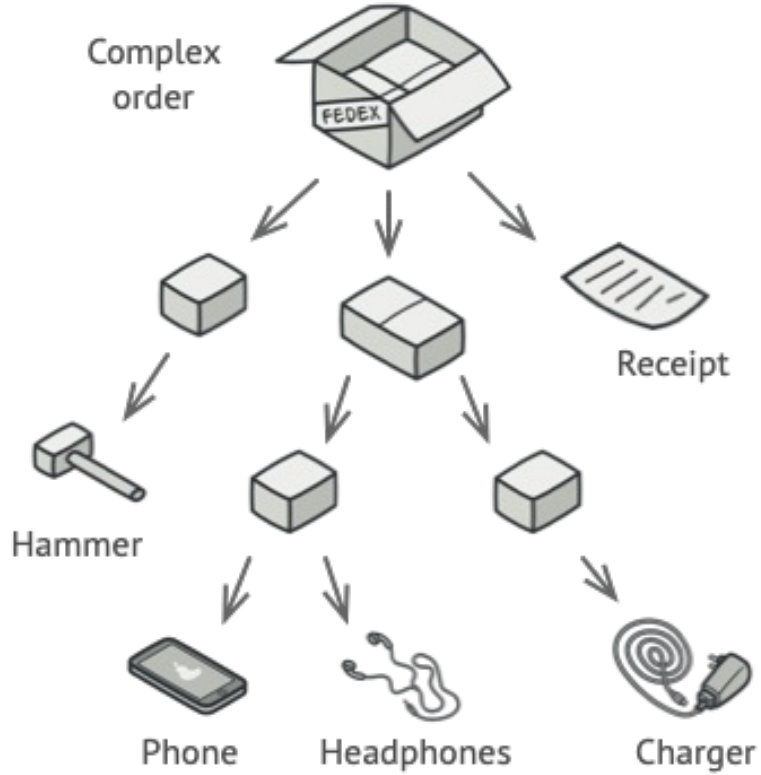
- Applicability
  - When a class in your program should have just a single instance available to all clients
    - E.g., a single database object shared by different parts of the program
  - When you need stricter control over global variables
    - E.g., configuration, logger, catching,…

Singleton

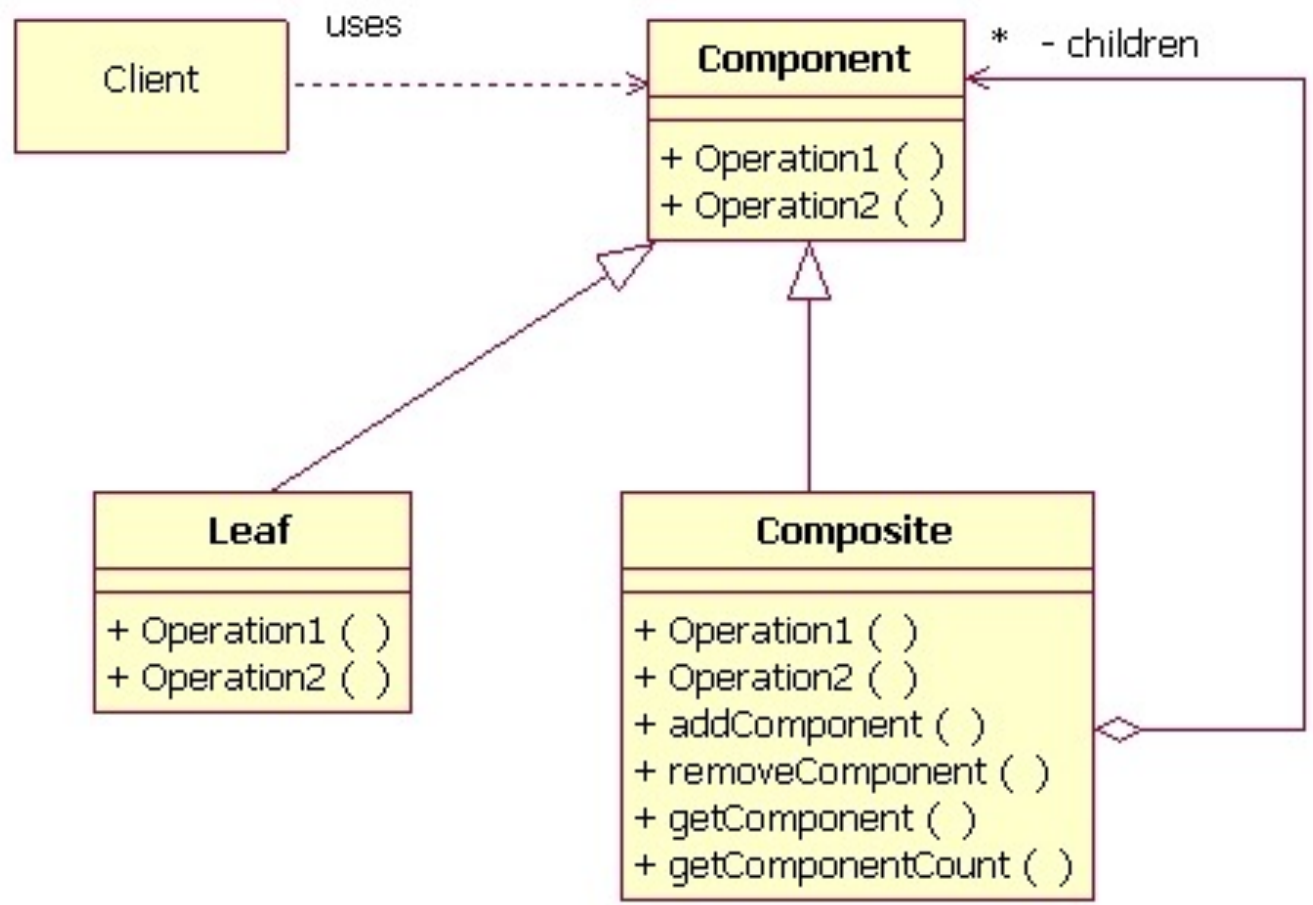
- Pros
  - You can be sure that a class has only a single instance
  - You gain a global access point to that instance
  - The singleton object is initialized only when it’s requested for the first time
- Cons
  - Violates the Single Responsibility Principle
  - Requires special treatment in a multithreaded environment

Composite

- Intent
  - Compose objects into tree structures and then work with these structures as if they were individual objects
- Problem
  - Using the Composite pattern makes sense only when the core model of your app can be represented as a tree.
- Solution
  - The Composite pattern suggests that you work with Products and Boxes through a common interface which declares a method for calculating the total price.



Composite


Client
uses
Component
+ Operation1 ( )
+ Operation2 ( )
* - children
Leaf
+ Operation1 ( )
+ Operation2 ( )
Composite
+ Operation1 ( )
+ Operation2 ( )
+ addComponent ( )
+ removeComponent ( )
+ getComponent ( )
+ getComponentCount ( )

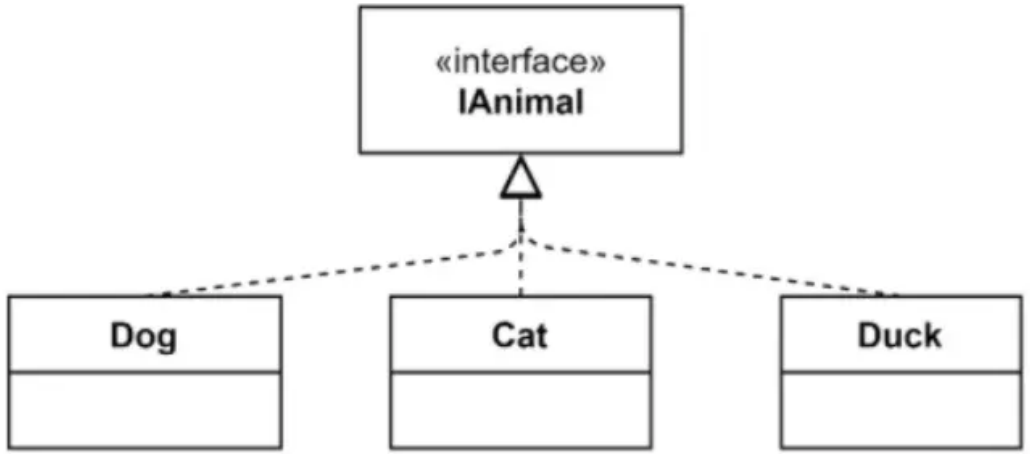
Composite

- Applicability
  - Use the Composite pattern when you have to implement a tree-like object structure.
  - Use the pattern when you want the client code to treat both simple and complex elements uniformly.

Factory Method

- Factory Method is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.

Factory Method

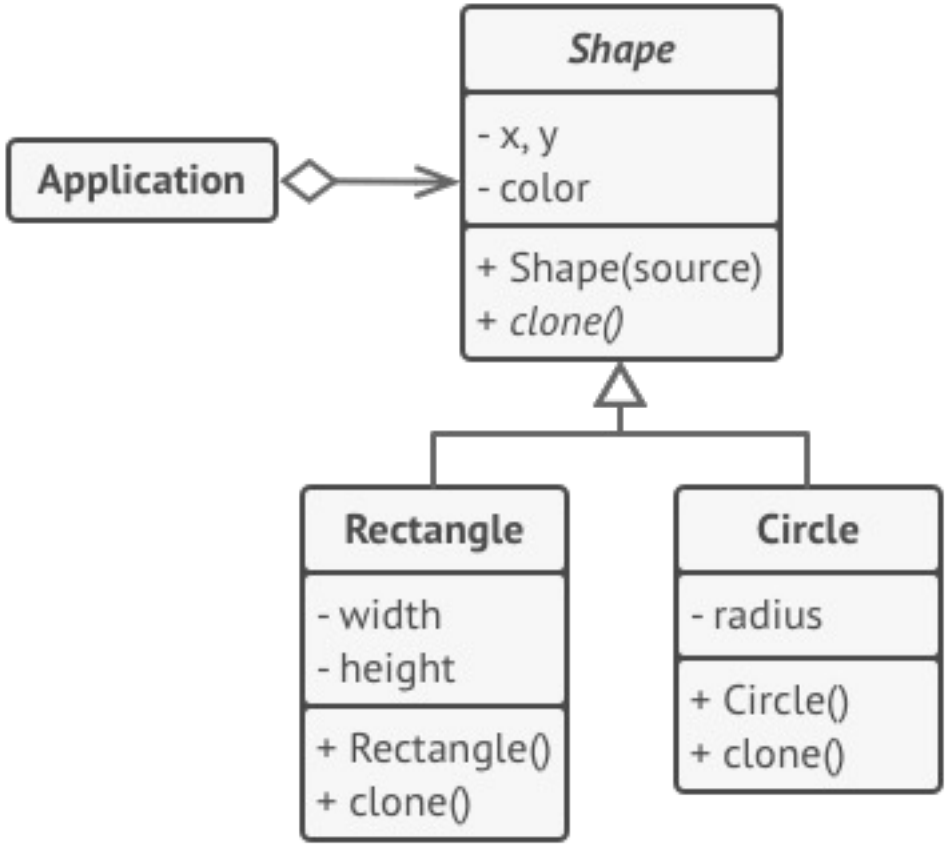



```
public class AnimalFactory {
    public static IAnimal CreateAnimal(AnimalType type)
    {
        IAnimal animal = null;
        switch (type) {
            case AnimalType.Cat:
                animal = new Cat();
                break;
            case AnimalType.Dog:
                animal = new Dog();
                break;
            case AnimalType.Duck:
                animal = new Duck();
                break;
        }
        return animal;
    }
}
```

Prototype

- Intent
  - Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes.
- Problem
  - Say you have an object, and you want to create an exact copy of it. How would you do it?
- Solution
  - The Prototype pattern delegates the cloning process to the actual objects that are being cloned.

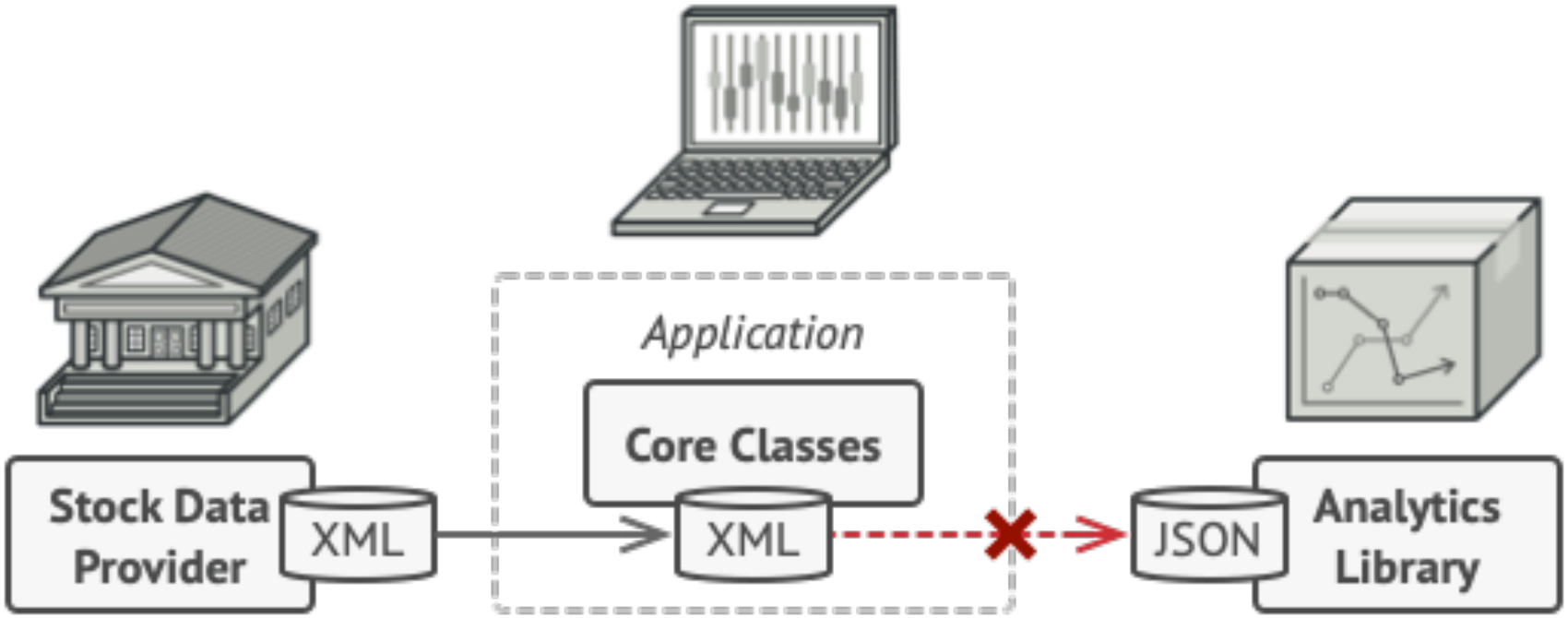
Prototype


Shape
- x, y
- color
+ Shape(source)
+ clone()
Application
Rectangle
- width
- height
+ Rectangle()
+ clone()
Circle
- radius
+ Circle()
+ clone()

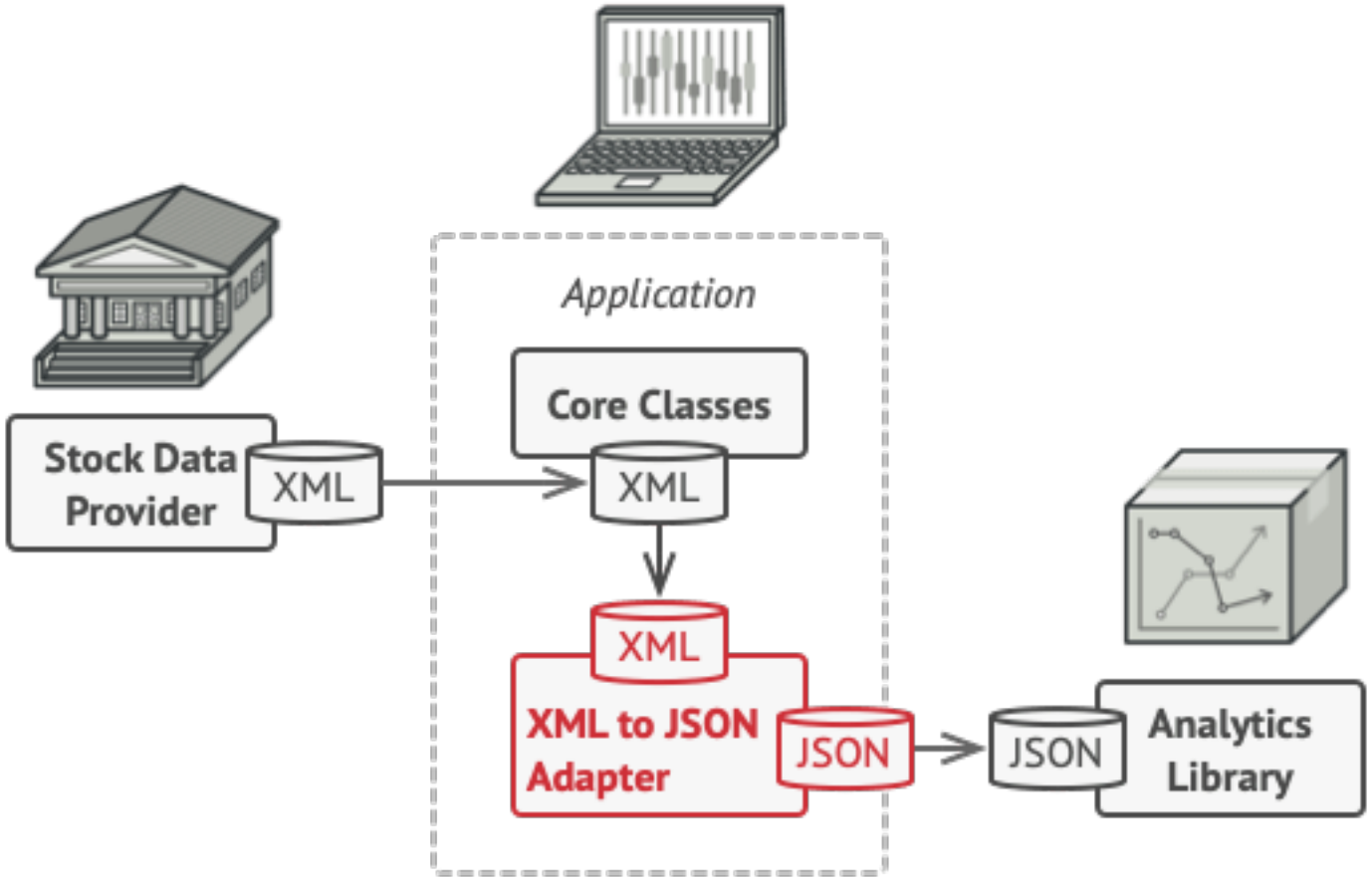
Adapter

- Intent
  - Adapter is a structural design pattern that allows objects with incompatible interfaces to collaborate.
- Problem
  - Imagine that you're creating a stock market monitoring app. The app downloads the stock data from multiple sources in XML format and then displays nice-looking charts and diagrams for the user.
- Solution
  - You can create an adapter. This is a special object that converts the interface of one object so that another object can understand it.

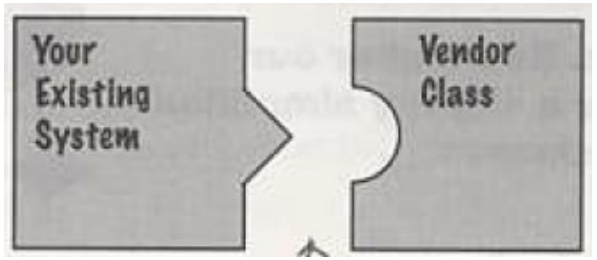
Adapter

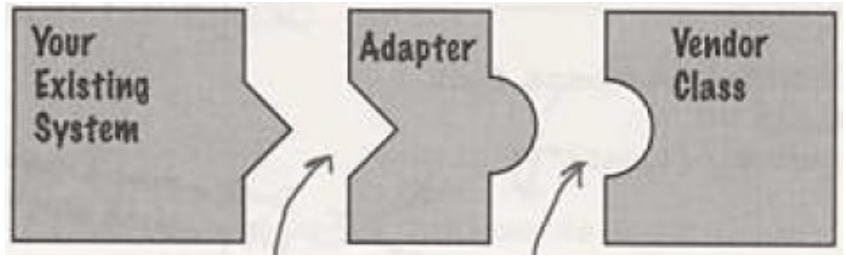

Application
Core Classes
Stock Data Provider
XML
XML
JSON
Analytics Library

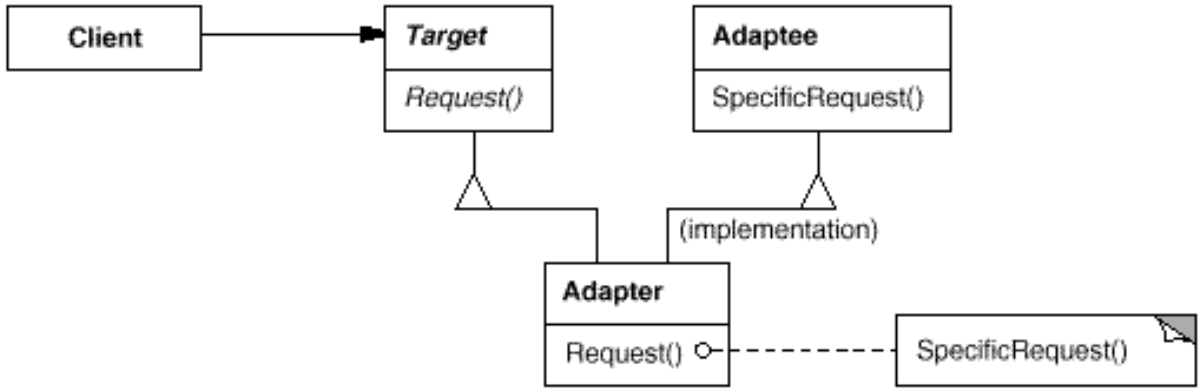
Adapter

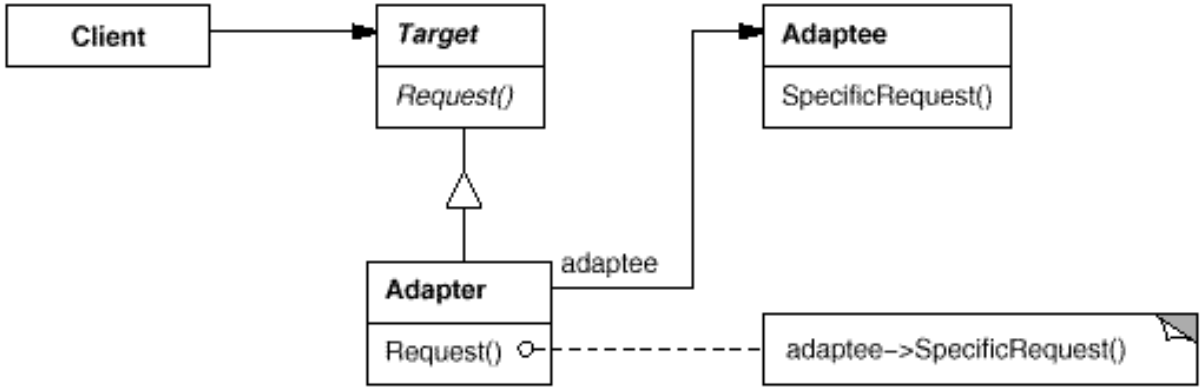

Application
Core Classes
Stock Data Provider
XML
XML
XML
XML to JSON Adapter
JSON
JSON
Analytics Library

Adapter


Your
Existing
System
Vendor
Class


Your
Existing
System
Adapter
Vendor
Class


Client
Target
Request()
Adaptee
SpecificRequest()
(implementation)
Adapter
Request()
SpecificRequest()


Client
Target
Request()
Adaptee
SpecificRequest()
adaptee
Adapter
Request()
adaptee->SpecificRequest()

Exercise 8.1

Apply OOP and Design Patterns to implement a computer file system (FS). The (FS) on a computer is organized into Disks. Each Disk can contain Folders, Files, and Shortcuts. A Folder can contain Files, Shortcuts, and subfolders. The main attributes of Disk, Folder, File, and Shortcut include a name. Additionally:

- Disk has information about its capacity.
- File has information about its size and type (binary or text).
- Shortcut has information about the path to the source (the Folder or File that the shortcut points to).

a. Implement the classes, including the FS, Disk, Folder, File, Shortcut, and any other necessary classes or interfaces.

b. Implement the getFreeSpace() method to calculate the available space on a disk. Assume that the size of a Shortcut is 1KB. The size of a Folder is the sum of the sizes of all files, shortcuts, and subfolders it contains.